

Rateforge CLI Demonstration (Windows PowerShell)

1. Token Bucket

Single Request

```
$r = Invoke-WebRequest `
    -Uri "http://localhost:3000/demo/token-bucket" `
    -Headers @{ "X-API-Key" = "demo-pro" }

Write-Host "Status      :" $r.StatusCode
Write-Host "Limit       :" $r.Headers["X-RateLimit-Limit"]
Write-Host "Remaining    :" $r.Headers["X-RateLimit-Remaining"]
Write-Host "Reset        :" $r.Headers["X-RateLimit-Reset"]
```

Continuous Requests

```
1..15 | ForEach-Object {
    try {
        $r = Invoke-WebRequest `
            -Uri "http://localhost:3000/demo/token-bucket" `
            -Headers @{ "X-API-Key" = "demo-pro" }

        Write-Host ("Request {0,-2} | Status={1} | Remaining={2}" -f `
            $_,
            $r.StatusCode,
            $r.Headers["X-RateLimit-Remaining"])
    }
    catch {
        Write-Host ("Request {0,-2} | Status={1}" -f `
            $_,
            [int]$_ .Exception.Response.StatusCode)
        break
    }
}
```

Wait for Token Refill

```
Write-Host "Waiting 5 seconds..."
Start-Sleep -Seconds 5
```

```
$r = Invoke-WebRequest `
    -Uri "http://localhost:3000/demo/token-bucket" `
    -Headers @{ "X-API-Key" = "demo-pro" }
```

```
Write-Host "Status      :" $r.StatusCode
Write-Host "Remaining  :" $r.Headers["X-RateLimit-Remaining"]
```

2. Sliding Window

Single Request

```
$r = Invoke-WebRequest `
    -Uri "http://localhost:3000/demo/sliding-window" `
    -Headers @{ "X-API-Key" = "demo-pro" }
```

```
Write-Host "Status      :" $r.StatusCode
Write-Host "Limit       :" $r.Headers["X-RateLimit-Limit"]
Write-Host "Remaining  :" $r.Headers["X-RateLimit-Remaining"]
Write-Host "Reset      :" $r.Headers["X-RateLimit-Reset"]
```

Multiple Requests

```
1..15 | ForEach-Object {
    try {
        $r = Invoke-WebRequest `
            -Uri "http://localhost:3000/demo/sliding-window" `
            -Headers @{ "X-API-Key" = "demo-pro" }

        Write-Host ("Request {0,-2} | Status={1} | Remaining={2}" -f `
            $_,
            $r.StatusCode,
            $r.Headers["X-RateLimit-Remaining"])
    }
    catch {
        Write-Host ("Request {0,-2} | Status={1}" -f `
            $_,
            [int]$_ .Exception.Response.StatusCode)
        break
    }
}
```

```
}
```

Wait for Window Expiration

```
Write-Host "Waiting for sliding window to expire..."
```

```
Start-Sleep -Seconds 5
```

```
$r = Invoke-WebRequest `
    -Uri "http://localhost:3000/demo/sliding-window" `
    -Headers @{ "X-API-Key" = "demo-pro" }
```

```
Write-Host "Status      :" $r.StatusCode
```

```
Write-Host "Remaining  :" $r.Headers["X-RateLimit-Remaining"]
```

3. Leaky Bucket

Single Request

```
$r = Invoke-WebRequest `
    -Uri "http://localhost:3000/demo/leaky-bucket" `
    -Headers @{ "X-API-Key" = "demo-pro" }
```

```
Write-Host "Status      :" $r.StatusCode
```

```
Write-Host "Limit       :" $r.Headers["X-RateLimit-Limit"]
```

```
Write-Host "Remaining  :" $r.Headers["X-RateLimit-Remaining"]
```

```
Write-Host "Reset       :" $r.Headers["X-RateLimit-Reset"]
```

Queue Filling Demo

```
1..15 | ForEach-Object {
    try {
        $r = Invoke-WebRequest `
            -Uri "http://localhost:3000/demo/leaky-bucket" `
            -Headers @{ "X-API-Key" = "demo-pro" }

        Write-Host ("Request {0,-2} | Status={1} | Remaining={2}" -f `
            $_,
            $r.StatusCode,
            $r.Headers["X-RateLimit-Remaining"])
    }
    catch {
```

```
Write-Host ("Request {0,-2} | Status={1}" -f `
    $_,
    [int]$_ .Exception.Response.StatusCode)
break
}
}
```

Wait for Queue Drain

```
Write-Host "Waiting 5 seconds for bucket to drain..."
Start-Sleep -Seconds 5
```

```
$r = Invoke-WebRequest `
    -Uri "http://localhost:3000/demo/leaky-bucket" `
    -Headers @{ "X-API-Key" = "demo-pro" }
```

```
Write-Host "Status      :" $r.StatusCode
Write-Host "Remaining   :" $r.Headers["X-RateLimit-Remaining"]
```

Health Check

```
Invoke-RestMethod `
    -Uri "http://localhost:3000/health"
```

Redis Health Check

```
Invoke-RestMethod `
    -Uri "http://localhost:3000/health/redis"
```

Docker Redis Check

```
docker ps
```

Redis Ping

```
docker exec -it rateforge-redis redis-cli ping
```

Expected:

PONG

View Redis Keys

```
docker exec -it rateforge-redis redis-cli KEYS "**"
```

View Token Bucket State

```
docker exec -it rateforge-redis redis-cli GET "demo-pro:tokens"
```

```
docker exec -it rateforge-redis redis-cli GET "demo-pro:lastRefill"
```

View Sliding Window State

```
docker exec -it rateforge-redis redis-cli ZRANGE "demo-pro" 0 -1 WITHSCORES
```

View Leaky Bucket State

```
docker exec -it rateforge-redis redis-cli GET "demo-pro:queue"
```

```
docker exec -it rateforge-redis redis-cli GET "demo-pro:lastLeak"
```

This collection gives you a complete reference for demonstrating all three rate-limiting algorithms, verifying the backend health, and inspecting the Redis state during an interview.